

Java Backend Interview Handbook

OOP · Core Java · Collections · Streams · Spring Boot · Design Patterns

✦ Java 8+ Features

✦ JUnit 5

✦ Git & Maven

✦ SOLID Principles

✦ Spring Framework

✦ Coding Exercises

Comprehensive preparation guide · Java 8-21 · Spring Boot 3.x

Table of Contents

Chapter 1 — OOP Concepts

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Interface vs Abstract Class
- Inner Classes

Chapter 2 — Core Java Essentials

- String & String Pool
- Static Keyword
- Final Keyword
- Serialization
- JMM
- Generics

Chapter 3 — Exception Handling

- try-catch-finally
- throw vs throws
- Custom Exceptions
- Try-with-resources

Chapter 4 — Collections Framework

- List / Set / Queue / Map
- ArrayList vs LinkedList
- HashMap Internals
- TreeSet / Comparator
- Concurrent Collections

Chapter 5 — Java 8+ Features

- Lambda & Functional Interfaces
- Streams API
- Optional
- Method References
- Records & Sealed Classes

Chapter 6 — Git

- Core Commands
- merge vs rebase
- fetch vs pull

Chapter 7 — Maven

- Lifecycle
- pom.xml
- jar vs war

Chapter 8 — Design Principles

- SOLID

- DRY / KISS / YAGNI

Chapter 9 — Design Patterns

- Singleton
- Factory
- Abstract Factory
- Decorator
- Strategy

Chapter 10 — Spring Framework

- IoC & DI
- Bean Scopes
- ApplicationContext
- Spring Boot Auto-config

Chapter 11 — JUnit 5

- Annotations
- Assertions
- Parameterized Tests
- Exception Testing

Chapter 12 — Stream Coding Exercises

- Employee Problems
- String Problems
- Number Problems

■ Chapter 1 — OOP Concepts

1.1 The Four Pillars

Q1. What are the four OOP concepts? Explain each.

Encapsulation — Bundling data (fields) and behaviour (methods) together and restricting direct access via access modifiers. Promotes data hiding and controlled access through getters/setters.

Abstraction — Hiding implementation details and exposing only what is necessary. Achieved via abstract classes and interfaces in Java.

Inheritance — A child class acquires properties/behaviour of a parent class using the *extends* keyword, enabling code reuse.

Polymorphism — 'Many forms'. The same method name behaves differently depending on context. Two types: Compile-time (method overloading) and Runtime (method overriding).

```
// Encapsulation
public class BankAccount {
    private double balance;          // hidden field
    public double getBalance() { return balance; }
    public void deposit(double amt) {
        if (amt > 0) balance += amt; // controlled mutation
    }
}

// Inheritance + Runtime Polymorphism
abstract class Shape {
    abstract double area();          // abstraction
}
class Circle extends Shape {
    private final double r;
    Circle(double r) { this.r = r; }
    @Override
    public double area() { return Math.PI * r * r; }
}
class Square extends Shape {
    private final double s;
    Square(double s) { this.s = s; }
    @Override
    public double area() { return s * s; }
}

// Polymorphic call
Shape sh = new Circle(5);
System.out.println(sh.area()); // Circle's area()
```

■ OOP is not just theory — every Spring bean, JPA entity, and REST controller you write uses all four pillars.

Q2. Interface vs Abstract Class — when to use which?

An **abstract class** can have state (fields), constructors, and concrete methods. Use it when classes share common state or partial implementations.

An **interface** (Java 8+) can have default and static methods but no instance state. Use it for defining a contract that unrelated classes can fulfil.

Key rule: Java supports *multiple interface implementation* but only *single class inheritance*.

```
// Abstract class – shared state & partial impl
abstract class Vehicle {
    protected int speed;           // state
    abstract void fuelType();
    void accelerate() { speed += 10; } // concrete
}

// Interface – pure contract
interface Flyable {
    void fly();
    default void land() {           // default method (Java 8+)
        System.out.println("Landing...");
    }
}

class FlyingCar extends Vehicle implements Flyable {
    @Override void fuelType() { System.out.println("Electric"); }
    @Override public void fly() { System.out.println("Flying!"); }
}
```

■ Prefer interfaces for API contracts; use abstract classes when you need shared template logic.

Q3. If two interfaces have the same default method, how do you resolve the conflict?

The implementing class **MUST** override the conflicting default method. You can explicitly call either interface's version using **InterfaceName.super.method()**.

```
interface A {
    default void greet() { System.out.println("Hello from A"); }
}

interface B {
    default void greet() { System.out.println("Hello from B"); }
}

class C implements A, B {
    @Override
    public void greet() {
        A.super.greet(); // explicitly call A's version
        // or B.super.greet();
        System.out.println("Hello from C");
    }
}
```

■ Compiler forces the override — it will not compile without it.

Q4. What are the rules for method overriding?

- Method signature (name + params) must be identical.
- Return type must be same or **covariant** (a subtype of parent's return type).
- Access modifier must be same or **more permissive** (e.g., protected → public).
- Cannot throw **new or broader checked exceptions**.
- Cannot override **static, final, or private** methods.
- Constructors cannot be overridden.

```
class Animal {
    protected Animal create() { return new Animal(); }
}
class Dog extends Animal {
    @Override
    public Dog create() { return new Dog(); } // covariant return type OK
}
```

Q5. Compile-time vs Runtime Polymorphism

Compile-time (Static): Method Overloading — same name, different parameter list. Resolved at compile time.

Runtime (Dynamic): Method Overriding — child class provides a specific impl. JVM uses virtual dispatch at runtime.

```
// Compile-time
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; } // overloaded
}

// Runtime
class Animal { void sound() { System.out.println("..."); } }
class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}
Animal a = new Cat();
a.sound(); // prints "Meow" - resolved at runtime
```

Q6. Types of Inner Classes in Java

Static nested class: declared static inside outer class. Can be created without outer instance.

Non-static inner class: has implicit reference to outer instance.

Local class: defined inside a method block.

Anonymous class: unnamed class instantiated inline, typically used for one-off interface implementations.

```
// Static nested
class Outer {
    static class StaticNested { void show() { System.out.println("Static"); } }

    // Non-static inner
    class Inner { void show() { System.out.println("Inner"); } }

    void method() {
        // Local class
        class Local { void show() { System.out.println("Local"); } }
        new Local().show();

        // Anonymous class
        Runnable r = new Runnable() {
            @Override public void run() { System.out.println("Anonymous"); }
        };
        r.run();
    }
}

// Usage
Outer.StaticNested sn = new Outer.StaticNested();
Outer.Inner inner = new Outer().new Inner();
```

■ Chapter 2 — Core Java Essentials

2.1 String & String Pool

Q7. What is the String Constant Pool? How does it work?

The **String Constant Pool (SCP)** is a special memory area in the heap (Java 8+, moved from PermGen). When you create a string literal, JVM checks the pool first; if found, it returns the reference — no new object is created. String created with **new** always creates a new heap object outside the pool. Use **intern()** to add it to the pool.

```
String s1 = "hello";           // added to SCP
String s2 = "hello";           // reused from SCP -> s1 == s2 is TRUE
String s3 = new String("hello"); // new heap object -> s1 == s3 is FALSE
String s4 = s3.intern();        // s4 points to SCP -> s1 == s4 is TRUE

System.out.println(s1 == s2); // true
System.out.println(s1 == s3); // false
System.out.println(s1 == s4); // true
System.out.println(s1.equals(s3)); // true (content comparison)
```

■ Always use `.equals()` for String content comparison, never `==`

Q8. String vs StringBuilder vs StringBuffer

String: Immutable. Every modification creates a new object. Thread-safe by nature (immutable).

StringBuilder: Mutable, NOT thread-safe. Faster for single-threaded use. Preferred in loops.

StringBuffer: Mutable, thread-safe (synchronized methods). Slower than StringBuilder.

```
// String – creates many intermediate objects
String result = "";
for (int i = 0; i < 1000; i++) result += i; // BAD: 1000 new String objects

// StringBuilder – single mutable buffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.append(i);
String result2 = sb.toString();           // GOOD

// StringBuffer – thread-safe
StringBuffer sbf = new StringBuffer();
sbf.append("thread").append("-safe");
```

Q9. hashCode() and equals() contract

- If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` MUST be true.
- The reverse is not required (hash collisions are allowed).
- Always override both together. Failing to do so breaks HashMap, HashSet, etc.

```
class Employee {
    private int id;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee e)) return false;
        return id == e.id && Objects.equals(name, e.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}
```

2.2 Static Keyword

Q10. Explain static variables, static methods, and static blocks. Can static methods be overridden?

Static variable: Class-level, shared among all instances. Memory allocated once in method area.

Static method: Belongs to class, not instance. Cannot access non-static members directly.

Static block: Executes once when class is loaded. Used for one-time setup.

Overriding: Static methods CANNOT be overridden — they are hidden (method hiding). The call is resolved at compile time based on the reference type.

```
class Parent {
    static void display() { System.out.println("Parent static"); }
}
class Child extends Parent {
    static void display() { System.out.println("Child static"); }
}

Parent obj = new Child();
obj.display();           // prints "Parent static" – NOT polymorphic!
Parent.display();        // prints "Parent static"
Child.display();         // prints "Child static"

// Static block example
class Config {
    static final String DB_URL;
    static {
        DB_URL = System.getenv("DB_URL"); // runs once at class load
        System.out.println("Config initialized");
    }
}
```

■ *Static method hiding vs overriding: the reference type, not the object type, determines which static method runs.*

2.3 Final Keyword

Q11. Explain final variable, final method, final class. Difference: final vs finally vs finalize

final variable: Value cannot be changed after assignment (primitive) or reference cannot change (object contents may change).

final method: Cannot be overridden by subclasses.

final class: Cannot be subclassed (e.g., String, Integer).

finally: Block in exception handling that always executes.

finalize(): Deprecated method called by GC before object destruction. Don't rely on it.

```
// final variable
final int MAX = 100;
// MAX = 200; // compilation error

// final method
class Base {
    final void compute() { System.out.println("Cannot override"); }
}

// final class
final class Immutable { }
// class Sub extends Immutable {} // error!

// finally
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("Always executes");
}
```

2.4 Serialization

Q12. What is Serialization and Deserialization? What is serialVersionUID? What happens without it?

Serialization: Converting a Java object to a byte stream for persistence or transmission.

Deserialization: Reconstructing the object from the byte stream.

serialVersionUID: A unique identifier for the class version. JVM uses it to verify compatibility during deserialization.

Without it, JVM auto-generates one based on class structure. If the class changes, the auto-generated UID changes, causing an **InvalidClassException** on deserialization of old data.

transient keyword excludes a field from serialization (e.g., passwords).

```
import java.io.*;

class Employee implements Serializable {
    private static final long serialVersionUID = 1L; // always declare this!
    private String name;
    private transient String password; // NOT serialized

    Employee(String name, String password) {
        this.name = name; this.password = password;
    }
    @Override public String toString() {
        return "Employee{name=" + name + ", password=" + password + "}";
    }
}

// Serialize
try (ObjectOutputStream oos = new ObjectOutputStream(
    new FileOutputStream("emp.ser"))) {
    oos.writeObject(new Employee("Alice", "secret"));
}

// Deserialize
try (ObjectInputStream ois = new ObjectInputStream(
    new FileInputStream("emp.ser"))) {
    Employee e = (Employee) ois.readObject();
    System.out.println(e); // Employee{name=Alice, password=null}
}
```

■ *password is null after deserialization because it was transient.*

2.5 Java Memory Model (JMM)

Q13. Explain different memory areas in JVM: Heap, Stack, Metaspace

Heap: Shared memory area. All objects and instance variables live here. Managed by GC. Has Young Gen (Eden, S0, S1) and Old Gen.

Stack: Per-thread memory. Stores method frames, local variables, and references. LIFO structure. `StackOverflowError` on deep recursion.

Metaspace (Java 8+, replaced PermGen): Stores class metadata, static variables, and interned strings. Grows dynamically in native memory.

PC Register: Per-thread. Holds address of current instruction.

Native Method Stack: For native (JNI) methods.

2.6 Generics & Type Erasure

Q14. What are Generics? What is Type Erasure?

Generics: Enable type-safe collections and methods by parameterizing types at compile time.

Type Erasure: Generics exist only at compile time. At runtime, type parameters are erased and replaced with `Object` (or bounds). This is for backward compatibility.

```
// Generic class
class Box<T> {
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}
Box<String> strBox = new Box<>("Hello");
// strBox.value is type-checked at compile time; at runtime it's just Object

// Wildcard – accept any subtype of Number
double sum(List<? extends Number> list) {
    return list.stream().mapToDouble(Number::doubleValue).sum();
}

// Bounded type parameter
<T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
```

■ Chapter 3 — Exception Handling

Q15. Exception hierarchy and types in Java

Throwable is the root. It has two subclasses:

Error: JVM-level issues. Unrecoverable (OutOfMemoryError, StackOverflowError). Don't catch these.

Exception: Two sub-categories:

- **Checked exceptions** (compile-time): Must be caught or declared. E.g., IOException, SQLException.
- **Unchecked exceptions** (runtime): Extend RuntimeException. E.g., NullPointerException, IllegalArgumentException.

```
// Checked exception – must handle
void readFile(String path) throws IOException {
    Files.readAllLines(Path.of(path));
}

// Unchecked exception – optional handling
void divide(int a, int b) {
    if (b == 0) throw new ArithmeticException("Cannot divide by zero");
    System.out.println(a / b);
}

// Custom checked exception
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}

// Custom unchecked exception
class InvalidAgeException extends RuntimeException {
    InvalidAgeException(String msg) { super(msg); }
}
```

Q16. try-catch-finally: What happens when both try AND finally have return statements, and try throws an exception?

Rule: **finally always executes** (even if try has a return statement). If finally also has a return statement, it **overrides** the try/catch return value.

Execution order: try (exception thrown) → catch → finally

If finally throws an exception, the original exception is lost.

```
// Scenario 1: try throws, catch returns, finally returns
static int test() {
    try {
        int x = 1 / 0;           // ArithmeticException thrown
        return 1;               // skipped
    } catch (Exception e) {
        System.out.println("catch");
        return 2;               // would return 2...
    } finally {
        System.out.println("finally");
        return 3;               // ...but this overrides! Returns 3
    }
}

// Output: catch, finally, returns 3

// try-with-resources (auto-closes resources)
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} // br.close() called automatically, even if exception thrown
```

■ Avoid return statements in finally blocks — it's confusing and swallows exceptions.

Q17. throw vs throws — difference with example

throw: Used inside method body to explicitly throw an exception instance.

throws: Used in method signature to declare that the method may throw a checked exception — caller must handle it.

```
// throw – explicit exception
void validateAge(int age) {
    if (age < 0) throw new IllegalArgumentException("Age cannot be negative: " + age);
}

// throws – declaration on method signature
void readConfig(String path) throws IOException, FileNotFoundException {
    // caller must catch IOException or declare it too
    Files.readString(Path.of(path));
}

// throw vs throws together
void process(String input) throws DataFormatException {
    if (input == null) throw new DataFormatException("Input is null");
}
```

■ Chapter 4 — Collections Framework

4.1 Collections Hierarchy

Q18. What is the Collections Framework? Explain key interfaces.

Iterable → **Collection** → List, Set, Queue, Deque

List: Ordered, allows duplicates. Implementations: ArrayList, LinkedList, Vector.

Set: No duplicates. Implementations: HashSet ($O(1)$), LinkedHashSet (insertion order), TreeSet (sorted, $O(\log n)$).

Queue: FIFO. Implementations: LinkedList, PriorityQueue, ArrayDeque.

Map: Key-value pairs. NOT part of Collection interface. Implementations: HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap.

Q19. ArrayList vs LinkedList — internal implementation and when to use each

ArrayList: Backed by a dynamic array. $O(1)$ for get/set. $O(n)$ for add/remove in middle. Preferred for random access.

LinkedList: Doubly linked list. $O(n)$ for get (must traverse). $O(1)$ for add/remove at head/tail. Preferred for frequent insertions/deletions.

```
// ArrayList – fast random access
List<String> al = new ArrayList<>();
al.add("A"); al.add("B"); al.add("C");
String s = al.get(1); // O(1)

// LinkedList – fast add/remove at ends
LinkedList<String> ll = new LinkedList<>();
ll.addFirst("A"); // O(1)
ll.addLast("B"); // O(1)
ll.removeFirst(); // O(1)

// Time complexity comparison
// Operation | ArrayList | LinkedList
// -----|-----|-----
// get(i)    | O(1)    | O(n)
// add(end)  | O(1)*   | O(1)
// add(middle)| O(n)    | O(n) to find, O(1) to insert
// remove(i) | O(n)    | O(n)
```

Q20. How does HashMap work internally?

HashMap uses an array of buckets (Node[]). The bucket index = hash(key) & (capacity-1).

Each bucket is a **linked list**. In Java 8+, when a bucket has more than 8 entries, it converts to a **Red-Black Tree** ($O(\log n)$ lookup instead of $O(n)$).

Collision: When two keys hash to same bucket, they are chained.

Load factor (default 0.75): When size > capacity * loadFactor, the map **reshashes** (doubles capacity).

Default capacity: 16. Always power of 2.

```
// HashMap internals (conceptual)
// Key → hashCode() → index in array
// index = (n-1) & hash   where n = table.length

HashMap<String, Integer> map = new HashMap<>(16, 0.75f);
map.put("Alice", 30);    // "Alice".hashCode() → index → store Node
map.put("Bob", 25);
int age = map.get("Alice"); // same hash path → return value

// Java 8+ treeification
// If bucket has > 8 nodes && table.length >= 64:
//   LinkedList bucket → TreeNode (Red-Black Tree)
// Benefit: worst case O(n) → O(log n)
```

■ Always override both `hashCode()` AND `equals()` when using custom objects as `HashMap` keys.

Q21. HashSet vs TreeSet — internal differences

HashSet: Backed by `HashMap`. $O(1)$ add/contains/remove. No guaranteed order.

TreeSet: Backed by `TreeMap` (Red-Black Tree). $O(\log n)$ operations. Elements are stored in **natural sorted order** or via a `Comparator`.

LinkedHashSet: Backed by `LinkedHashMap`. $O(1)$ ops. Maintains **insertion order**.

```
// Custom sort in TreeSet using Comparator
TreeSet<String> ts = new TreeSet<>(Comparator.comparingInt(String::length)
                                           .thenComparing(Comparator.naturalOrder()));

ts.add("banana"); ts.add("apple"); ts.add("fig");
System.out.println(ts); // [fig, apple, banana]

// Comparable – natural ordering
class Student implements Comparable<Student> {
    int gpa;
    @Override public int compareTo(Student o) {
        return Double.compare(o.gpa, this.gpa); // descending
    }
}
```

Q22. ConcurrentHashMap vs HashMap vs SynchronizedMap

HashMap: Not thread-safe. Multiple threads can corrupt it.

Collections.synchronizedMap(): Wraps `HashMap` with a single mutex. Entire map locked on every operation — poor concurrency.

ConcurrentHashMap: Segment-level locking (Java 7) / CAS + synchronized on individual buckets (Java 8+). Multiple threads can read/write different buckets concurrently. Null keys/values NOT allowed.

```
// ConcurrentHashMap – high-concurrency scenarios
ConcurrentHashMap<String, Integer> chm = new ConcurrentHashMap<>();
chm.put("a", 1);
chm.putIfAbsent("b", 2);           // atomic
chm.computeIfAbsent("c", k -> 3);  // atomic compute

// Safe iteration – ConcurrentHashMap uses fail-safe iterator
// (iterates over snapshot, no ConcurrentModificationException)
for (Map.Entry<String, Integer> entry : chm.entrySet()) {
    System.out.println(entry.getKey() + "=" + entry.getValue());
}

// Fail-fast (HashMap) vs Fail-safe (ConcurrentHashMap)
Map<String, Integer> hm = new HashMap<>();
hm.put("x", 1);
Iterator<String> it = hm.keySet().iterator();
hm.put("y", 2); // ConcurrentModificationException on next it.next()
```

■ Prefer `ConcurrentHashMap` over `synchronizedMap` in multi-threaded code for much better throughput.

Q23. Comparable vs Comparator

Comparable: Interface in `java.lang`. Method: `compareTo(T o)`. Defines the object's **natural ordering**. Class itself implements it.

Comparator: Interface in `java.util`. Method: `compare(T o1, T o2)`. Defines **external/custom ordering**. Separation of concerns — ordering logic outside the class.

```
// Comparable – natural order (part of the class)
class Employee implements Comparable<Employee> {
    String name; int salary;
    @Override
    public int compareTo(Employee o) {
        return Integer.compare(this.salary, o.salary); // sort by salary asc
    }
}

// Comparator – multiple orderings externally
Comparator<Employee> byName = Comparator.comparing(e -> e.name);
Comparator<Employee> byDept = Comparator.comparing((Employee e) -> e.dept)
    .thenComparingInt(e -> e.salary);

List<Employee> emps = ...;
emps.sort(byName);
emps.sort(byDept);
```


■ Chapter 5 — Java 8+ Features

5.1 Lambda Expressions & Functional Interfaces

Q24. What is a Lambda Expression? Lambda vs Anonymous Class

A lambda is a concise way to represent a **functional interface** (an interface with exactly one abstract method — SAM). It has no name, no class declaration.

Syntax: (params) -> expression or (params) -> { statements; }

Lambda vs Anonymous class: Lambda is more concise, cannot have state (no fields), and **this** refers to enclosing class. Anonymous class can have state and **this** refers to itself.

```
// Anonymous class
Runnable r1 = new Runnable() {
    @Override public void run() { System.out.println("Anonymous"); }
};

// Lambda equivalent
Runnable r2 = () -> System.out.println("Lambda");

// Functional Interfaces
Predicate<Integer> isEven = n -> n % 2 == 0;
Function<String, Integer> len = String::length;
Consumer<String> printer = System.out::println;
Supplier<List<String>> listFactory = ArrayList::new;

// Composing predicates
Predicate<Integer> isPositive = n -> n > 0;
Predicate<Integer> isEvenPositive = isEven.and(isPositive);
System.out.println(isEvenPositive.test(4)); // true
System.out.println(isEvenPositive.test(-2)); // false
```

■ Functional interfaces: Predicate, Function, Consumer, Supplier, BiFunction

Q25. Is an interface with equals() and hashCode() a Functional Interface? What if it extends Comparable?

Yes, it IS a functional interface if it has exactly one **abstract** method that is NOT from java.lang.Object.

equals() and hashCode() are Object methods — they don't count toward the SAM count. They are implicitly provided.

If it also extends Comparable, that adds compareTo() as an abstract method, making it **two** abstract methods — it is **no longer a functional interface**.

```
@FunctionalInterface
interface MyFunc {
    void execute();           // 1 abstract method – SAM
    boolean equals(Object o); // from Object – doesn't count
    int hashCode();           // from Object – doesn't count
}
// This IS a functional interface ✓

interface NotFunctional extends Comparable<NotFunctional> {
    void execute();           // abstract method 1
    // compareTo() from Comparable is abstract method 2
}
// This is NOT a functional interface ✗
```

5.2 Method References

Q26. Types of Method References with examples

1. **Static method reference:** `ClassName::staticMethod`
2. **Instance method of a particular object:** `instance::instanceMethod`
3. **Instance method of an arbitrary object of a type:** `ClassName::instanceMethod`
4. **Constructor reference:** `ClassName::new`

```
// 1. Static method reference
Function<String, Integer> parser = Integer::parseInt;
System.out.println(parser.apply("42")); // 42

// 2. Instance method of a specific object
String prefix = "Hello, ";
Function<String, String> greeter = prefix::concat;
System.out.println(greeter.apply("Alice")); // Hello, Alice

// 3. Instance method of arbitrary object (first param becomes receiver)
Comparator<String> comp = String::compareToIgnoreCase;
List<String> names = Arrays.asList("Bob", "alice", "Charlie");
names.sort(comp);

// 4. Constructor reference
Supplier<ArrayList<String>> listMaker = ArrayList::new;
List<String> list = listMaker.get();
```

5.3 Streams API

Q27. What is a Stream? Lazy evaluation? Parallel streams?

A **Stream** is a sequence of elements supporting sequential and parallel aggregate operations. It does NOT store data — it processes a source (collection, array, I/O).

Lazy evaluation: Intermediate operations (filter, map, sorted) are not executed until a terminal operation (collect, forEach, count) is invoked. Only the necessary elements are processed.

Parallel stream: Uses ForkJoinPool. Splits data across multiple threads. Use for CPU-bound, stateless, order-independent operations on large datasets.

```
// Lazy evaluation demonstration
Stream<String> stream = List.of("a","b","c","d")
    .stream()
    .filter(s -> { System.out.println("filter: " + s); return true; })
    .map(s -> { System.out.println("map: " + s); return s.toUpperCase(); });
// Nothing printed yet – no terminal operation

stream.findFirst(); // Only processes until first match

// Parallel stream
long count = IntStream.rangeClosed(1, 1_000_000)
    .parallel()
    .filter(n -> n % 2 == 0)
    .count(); // uses multiple CPU cores

// Common stream operations
List<String> result = Stream.of("foo","bar","baz","qux")
    .filter(s -> s.startsWith("b")) // intermediate (lazy)
    .map(String::toUpperCase)      // intermediate (lazy)
    .sorted()                      // intermediate (lazy)
    .collect(Collectors.toList()); // terminal – triggers execution
```

■ *flatMap() vs map(): map() transforms each element 1:1; flatMap() maps each element to a stream then flattens.*

5.4 Optional

Q28. What is Optional? orElse vs orElseGet. Common methods.

Optional is a container that may or may not hold a value. It eliminates explicit null checks and NullPointerExceptions.

orElse(T val): Always evaluates val (even if Optional has a value). val is a plain value.

orElseGet(Supplier): Evaluates Supplier only if Optional is empty. More efficient when the default value is expensive to compute.

```
Optional<String> opt = Optional.of("hello");
Optional<String> empty = Optional.empty();

// orElse – always evaluates the argument
String s1 = empty.orElse("default"); // "default"
String s2 = empty.orElse(expensiveMethod()); // expensiveMethod() called!

// orElseGet – lazy; Supplier called only if empty
String s3 = empty.orElseGet(() -> expensiveMethod()); // called only when needed
String s4 = opt.orElseGet(() -> expensiveMethod()); // NOT called

// map, flatMap, filter
Optional<Integer> length = opt.map(String::length); // Optional[5]
opt.filter(s -> s.startsWith("h"))
    .ifPresent(System.out::println); // prints "hello"

// orElseThrow
String val = opt.orElseThrow(() -> new RuntimeException("No value!"));
```

■ *Don't use Optional as method parameters or fields — use it only as return types.*

5.5 Records & Sealed Classes (Java 16+/17+)

Q29. What are Records? What are Sealed Classes?

Records (Java 16): Immutable data carriers. Compiler auto-generates constructor, getters, equals(), hashCode(), toString(). Cannot extend other classes.

Sealed Classes (Java 17): Restrict which classes can extend/implement them. Used with permits keyword. Enables exhaustive pattern matching.

```
// Record
record Point(int x, int y) {
    // Compact constructor for validation
    Point {
        if (x < 0 || y < 0) throw new IllegalArgumentException("Coords must be >= 0");
    }
    // Can add custom methods
    double distanceFromOrigin() { return Math.sqrt(x * x + y * y); }
}

Point p = new Point(3, 4);
System.out.println(p.x()); // 3 (auto-generated accessor)
System.out.println(p);      // Point[x=3, y=4]

// Sealed class
sealed interface Shape permits Circle, Rectangle, Triangle {}

record Circle(double radius) implements Shape {}
record Rectangle(double w, double h) implements Shape {}
final class Triangle implements Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base; this.height = height;
    }
}

// Pattern matching with sealed class (Java 21)
double area(Shape s) {
    return switch (s) {
        case Circle c    -> Math.PI * c.radius() * c.radius();
        case Rectangle r -> r.w() * r.h();
        case Triangle t   -> 0.5 * t.base * t.height;
    };
}
```

■ Chapter 6 — Git

Q30. What is Git? Why was it introduced? Daily-use commands.

Git is a **distributed version control system** created by Linus Torvalds in 2005 to manage Linux kernel development. It allows teams to collaborate, track history, and branch without a single point of failure.

```
# Daily workflow
git status                # show working tree status
git add .                 # stage all changes
git add src/Main.java     # stage specific file
git commit -m "feat: add login API"
git push origin feature/login

# Branching
git branch                # list local branches
git branch -a             # list ALL branches (local + remote)
git branch -r             # list remote branches only
git checkout -b feature/login # create and switch
git switch feature/login   # modern syntax

# Log and diff
git log --oneline --graph --all # visual branch history
git diff HEAD~1 HEAD           # diff last commit

# Undo
git restore src/Main.java      # discard unstaged changes
git reset HEAD~1              # undo last commit (keep changes staged)
git reset --hard HEAD~1        # undo last commit (discard changes) ■■

# Stash
git stash push -m "WIP login"
git stash pop                 # apply and drop last stash
```

Q31. git merge vs git rebase — difference, use cases, pros/cons

git merge: Integrates changes by creating a **merge commit**. Preserves complete history. Safe for shared branches.

git rebase: Re-applies commits on top of another base commit. Creates a **linear history**. Rewrites commit SHAs — dangerous on shared branches.

```
# Merge – preserves history, adds merge commit
git checkout main
git merge feature/login    # creates merge commit M

# History: A-B-C-M (M has two parents)

# Rebase – linear history
git checkout feature/login
git rebase main            # re-applies feature commits on top of main

# History: A-B-C-D'-E' (D' and E' are new commits)
# Then fast-forward merge:
git checkout main
git merge feature/login    # fast-forward, no merge commit

# Interactive rebase – squash/edit commits before PR
git rebase -i HEAD~3      # interactive rebase last 3 commits

# Golden rule: NEVER rebase commits that have been pushed to shared branches
# git fetch vs git pull
git fetch origin          # download remote changes, don't merge
git pull origin main      # fetch + merge (or fetch + rebase with --rebase)
```

■ Use merge for long-lived shared branches; use rebase to clean up your local feature branch before creating a PR.

Chapter 7 — Maven

Q32. What is Maven? Build lifecycle phases. mvn package vs mvn install.

Maven is a **build automation and dependency management tool**. It defines a standard project structure and lifecycle.

Default lifecycle phases (in order): validate → compile → test → package → verify → install → deploy

mvn package: Compiles, tests, and creates the JAR/WAR in /target. Does NOT install to local repo.

mvn install: All of package + copies artifact to local Maven repository (~/.m2). Other local projects can then depend on it.

```
<!-- pom.xml structure -->
<project>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>    <!-- jar (default) or war -->

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.2.0</version>
    </dependency>
    <!-- Test scope – not included in final artifact -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <!-- packaging: jar for standalone app, war for external servlet container -->
  <!-- Specify in <packaging> tag. Spring Boot defaults to jar -->
</project>

# How to check transitive dependencies
mvn dependency:tree
mvn dependency:tree -Dincludes=org.slf4j:slf4j-api

# Exclude transitive dependency
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

■ *jar = embedded server (Spring Boot), war = deployed to external Tomcat/JBoss. Set war and extend SpringBootServletInitializer for WAR.*

■ Chapter 8 — Design Principles

Q33. Explain each SOLID principle with a Java example

S — Single Responsibility Principle (SRP): A class should have only one reason to change. Separate concerns.

O — Open/Closed Principle (OCP): Open for extension, closed for modification. Add new behaviour without changing existing code.

L — Liskov Substitution Principle (LSP): Subtypes must be substitutable for their base types without altering correctness.

I — Interface Segregation Principle (ISP): Clients should not be forced to depend on interfaces they don't use. Prefer small, focused interfaces.

D — Dependency Inversion Principle (DIP): High-level modules should not depend on low-level modules. Both should depend on abstractions.

```
// S - SRP: separate order and notification concerns
class Order { void placeOrder() { /* order logic */ } }
class OrderNotifier { void sendConfirmation(Order o) { /* email logic */ } }

// O - OCP: new discount types without modifying DiscountCalculator
interface Discount { double apply(double price); }
class SeasonalDiscount implements Discount {
    public double apply(double p) { return p * 0.90; }
}
class LoyaltyDiscount implements Discount {
    public double apply(double p) { return p * 0.85; }
}
class DiscountCalculator {
    double calculate(double price, Discount d) { return d.apply(price); }
}

// L - LSP violation & fix
// Bad: Square extends Rectangle breaks setWidth/setHeight invariants
// Good: both implement Shape with area()

// I - ISP: don't force all machines to implement all methods
interface Printable { void print(); }
interface Scannable { void scan(); }
class SimplePrinter implements Printable { public void print() {} }
class MultiFunction implements Printable, Scannable {
    public void print() {} public void scan() {}
}

// D - DIP: depend on abstraction (UserRepository), not impl (MySQLUserRepository)
interface UserRepository { Optional<User> findById(long id); }
class UserService {
    private final UserRepository repo; // abstraction
    UserService(UserRepository repo) { this.repo = repo; } // injected
}
```

■ Chapter 9 — Design Patterns

Q34. Singleton Pattern — thread-safe implementation

Ensures a class has **only one instance** and provides a global access point. Used in: DB connection pool, Spring beans (default scope), Logger, Config.

```
// Thread-safe Singleton using enum (best practice)
enum DatabaseConnection {
    INSTANCE;
    private final Connection conn;
    DatabaseConnection() { conn = DriverManager.getConnection(DB_URL); }
    public Connection get() { return conn; }
}

// Double-checked locking (pre-Java 5 workaround, still seen in interviews)
class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) { // first check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) { // second check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

// Spring uses Singleton scope by default:
@Service // one instance per ApplicationContext
class MyService { }
```

■ *Spring beans are Singletons by default — which pattern is used when creating them? Singleton + Factory.*

Q35. Factory & Abstract Factory Pattern

Factory Method: Defines an interface for creating an object but lets subclasses decide which class to instantiate. Separates creation from usage.

Abstract Factory: Creates families of related objects without specifying concrete classes.

```
// Factory Method
interface Notification { void send(String message); }
class EmailNotification implements Notification {
    public void send(String msg) { System.out.println("Email: " + msg); }
}
class SMSNotification implements Notification {
    public void send(String msg) { System.out.println("SMS: " + msg); }
}
class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "EMAIL" -> new EmailNotification();
            case "SMS" -> new SMSNotification();
            default -> throw new IllegalArgumentException("Unknown: " + type);
        };
    }
}

// Abstract Factory – create families of related objects
interface Button { void render(); }
interface Checkbox { void render(); }

class WindowsButton implements Button { public void render() { System.out.println("Windows Button"); } }
class MacButton implements Button { public void render() { System.out.println("Mac Button"); } }

interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}
class WindowsFactory implements GUIFactory {
    public Button createButton() { return new WindowsButton(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}
```

Q36. Decorator Pattern

Attaches additional responsibilities to an object **dynamically**. Wraps the original object with decorator classes that add behaviour. Alternative to subclassing.

Real-world Java: `java.io.BufferedReader` wraps `FileReader`. Spring's `@Transactional` wraps your service with a proxy.

```
interface Coffee { double cost(); String description(); }

class Espresso implements Coffee {
    public double cost() { return 1.5; }
    public String description() { return "Espresso"; }
}

// Decorator base
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    CoffeeDecorator(Coffee c) { this.coffee = c; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee c) { super(c); }
    public double cost() { return coffee.cost() + 0.25; }
    public String description() { return coffee.description() + ", Milk"; }
}

class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(Coffee c) { super(c); }
    public double cost() { return coffee.cost() + 0.10; }
    public String description() { return coffee.description() + ", Sugar"; }
}

// Usage
Coffee myCoffee = new SugarDecorator(new MilkDecorator(new Espresso()));
System.out.println(myCoffee.description()); // Espresso, Milk, Sugar
System.out.println(myCoffee.cost());        // 1.85
```

Q37. Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Lets the algorithm vary independently from clients that use it.

Real-world: Different sort strategies, payment methods, compression algorithms.

```
// Strategy interface
interface SortStrategy { void sort(int[] arr); }

class BubbleSort implements SortStrategy {
    public void sort(int[] arr) { /* bubble sort impl */ }
}
class QuickSort implements SortStrategy {
    public void sort(int[] arr) { /* quicksort impl */ }
}

// Context – uses a strategy
class Sorter {
    private SortStrategy strategy;
    Sorter(SortStrategy s) { this.strategy = s; }
    void setStrategy(SortStrategy s) { this.strategy = s; }
    void sort(int[] arr) { strategy.sort(arr); }
}

// Usage – swap strategies at runtime
Sorter sorter = new Sorter(new BubbleSort());
sorter.sort(data);

sorter.setStrategy(new QuickSort()); // switch strategy
sorter.sort(data);

// With lambda (functional strategy)
SortStrategy reverse = arr -> {
    Arrays.sort(arr);
    // reverse in place...
};
sorter.setStrategy(reverse);
```

■ Chapter 10 — Spring Framework

Q38. What is IoC and Dependency Injection? Types of DI?

IoC (Inversion of Control): The Spring container controls object creation and lifecycle — NOT the developer. Control is inverted from code to the framework.

DI (Dependency Injection): The container injects dependencies into a bean rather than the bean creating them. Promotes loose coupling and testability.

Types of DI: Constructor injection (preferred), Setter injection, Field injection (@Autowired — avoid in tests).

```
// Constructor injection – preferred (immutable, testable)
@Service
public class OrderService {
    private final OrderRepository orderRepo;
    private final EmailService emailService;

    public OrderService(OrderRepository orderRepo, EmailService emailService) {
        this.orderRepo = orderRepo;
        this.emailService = emailService;
    }
}

// @Qualifier – resolve ambiguity when multiple beans of same type
@Service("fastEmail")
class FastEmailService implements EmailService {}

@Service("slowEmail")
class SlowEmailService implements EmailService {}

@Service
class NotificationService {
    @Autowired
    @Qualifier("fastEmail")
    private EmailService emailService;
}

// @Primary – default bean when no @Qualifier specified
@Primary
@Service
class PrimaryEmailService implements EmailService {}
```

Q39. Bean Scopes. ApplicationContext vs BeanFactory.

Singleton (default): One instance per ApplicationContext.

Prototype: New instance every time the bean is requested.

Request: One instance per HTTP request (web-aware contexts).

Session: One instance per HTTP session.

BeanFactory: Basic IoC container. Lazy initialization. Lower memory footprint.

ApplicationContext: Extends BeanFactory. Eager initialization of singletons. Supports i18n, events, AOP. Use this in production.

```
@Component
@Scope("prototype")
class PrototypeBean { }

@Component
@RequestScope
class RequestBean { } // new instance per HTTP request

// ApplicationContext
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
MyService svc = ctx.getBean(MyService.class);
```

Q40. Spring Boot — Auto-configuration, @SpringBootApplication, Profiles

@SpringBootApplication: Combines @Configuration + @EnableAutoConfiguration + @ComponentScan.

Auto-configuration: Spring Boot inspects classpath and auto-configures beans (e.g., if H2 is on classpath, configures an in-memory DataSource).

Profiles: Environment-specific config. Use @Profile('prod') on beans or application-prod.yml for properties.

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}

// application.yml
spring:
  profiles:
    active: prod # activate prod profile

---
spring:
  config:
    activate:
      on-profile: dev
  datasource:
    url: jdbc:h2:mem:devdb

---
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: jdbc:mysql://prod-server/mydb

// Profile-specific bean
@Configuration
@Profile("prod")
class ProdConfig {
    @Bean DataSource dataSource() { return new HikariDataSource(); }
}
```

■ Chapter 11 — JUnit 5 (Unit Testing)

Q41. All JUnit 5 annotations and assertions. Parameterized tests. Exception testing.

@Test: Marks a test method.

@BeforeEach / @AfterEach: Run before/after each test method.

@BeforeAll / @AfterAll: Run once before/after all tests in class (must be static).

@Disabled: Skip a test.

@DisplayName: Custom test name.

@ParameterizedTest: Run same test with different inputs.

@ExtendWith: Register extensions (e.g., MockitoExtension).


```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock OrderRepository orderRepo;
    @Mock EmailService emailService;
    @InjectMocks OrderService orderService;

    @BeforeAll
    static void globalSetup() { System.out.println("Before all tests"); }

    @BeforeEach
    void setUp() { /* runs before each test */ }

    @Test
    @DisplayName("Should place order successfully")
    void testPlaceOrder() {
        Order order = new Order("item1", 2);
        when(orderRepo.save(any())).thenReturn(order);

        Order placed = orderService.placeOrder(order);

        assertNotNull(placed);
        assertEquals("item1", placed.getItem());
        verify(emailService, times(1)).sendConfirmation(any());
    }

    @Test
    void testInsufficientStock() {
        // Exception testing
        assertThrows(InsufficientStockException.class, () -> {
            orderService.placeOrder(new Order("rare-item", 9999));
        });
    }

    @ParameterizedTest
    @ValueSource(strings = {"", " ", null})
    void testInvalidItemNames(String name) {
        assertThrows(IllegalArgumentException.class,
            () -> orderService.placeOrder(new Order(name, 1)));
    }

    @ParameterizedTest
    @CsvSource({"1,10.0,10.0", "2,10.0,20.0", "3,10.0,30.0"})
    void testOrderTotal(int qty, double price, double expected) {
        assertEquals(expected, orderService.calculateTotal(qty, price));
    }

    // Predicate testing
    @Test
    void testPredicate() {
        Predicate<Integer> isPositive = n -> n > 0;
        assertTrue(isPositive.test(5));
        assertFalse(isPositive.test(-1));
    }

    @AfterEach
    void tearDown() { /* cleanup */ }

    @AfterAll
    static void globalTearDown() { System.out.println("After all tests"); }
}
```

■ Chapter 12 — Stream Coding Exercises

12.1 Employee Stream Problems

Given Employee class: record Employee(String name, String dept, double salary, int experience) {}

Q42. 1. Filter employees with salary > threshold. 2. Sort names ascending.

```
record Employee(String name, String dept, double salary, int experience) {}

List<Employee> employees = List.of(
    new Employee("Alice", "IT", 90000, 6),
    new Employee("Bob", "HR", 45000, 3),
    new Employee("Charlie", "IT", 75000, 8),
    new Employee("Diana", "Finance", 120000, 10),
    new Employee("Eve", "HR", 52000, 2)
);

// 1. Filter by salary
double threshold = 70000;
List<Employee> highEarnings = employees.stream()
    .filter(e -> e.salary() > threshold)
    .collect(Collectors.toList());

// 2. Sort names in ascending order
List<String> sortedNames = employees.stream()
    .map(Employee::name)
    .sorted()
    .collect(Collectors.toList());

// 3. Uppercase all names
List<String> upperNames = employees.stream()
    .map(e -> e.name().toUpperCase())
    .collect(Collectors.toList());

// 4. Experience > 5, increase salary by 10%
List<Employee> updated = employees.stream()
    .filter(e -> e.experience() > 5)
    .map(e -> new Employee(e.name(), e.dept(), e.salary() * 1.1, e.experience()))
    .collect(Collectors.toList());

// 5. Highest salary per department
Map<String, Optional<Employee>> topPerDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        Collectors.maxBy(Comparator.comparingDouble(Employee::salary))
    ));

// 6. Average salary by department
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        Collectors.averagingDouble(Employee::salary)
    ));

// 7. Convert to Map<name, Employee>
Map<String, Employee> empMap = employees.stream()
    .collect(Collectors.toMap(Employee::name, e -> e));
```

12.2 String Stream Problems

Q43. String stream exercises from interviews

```
// Q: List<String> strArr = Arrays.asList("1,2,3","3,4,5","6,abc,7","8,9,10","10,2,4");
// Output: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1] (all valid integers, sorted desc, distinct)

List<String> strArr = Arrays.asList("1,2,3","3,4,5","6,abc,7","8,9,10","10,2,4");

List<Integer> result = strArr.stream()
    .flatMap(s -> Arrays.stream(s.split(","))) // split each CSV string
    .filter(s -> s.matches("\\d+"))           // keep only numeric strings
    .map(Integer::parseInt)                   // parse to int
    .distinct()                              // remove duplicates
    .sorted(Comparator.reverseOrder())       // sort descending
    .collect(Collectors.toList());
// [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

// Q: Integer num = 12345; convert to stream then sum digits
int num = 12345;
int digitSum = String.valueOf(num)
    .chars()                                // IntStream of char codes
    .map(c -> c - '0')                       // char to digit
    .sum();                                  // 15

// Q: Given ["Hello", "1wood", "23bottle", "abc"] – elements starting with digit
List<String> input = List.of("Hello", "1wood", "23bottle", "abc");
List<String> startsWithDigit = input.stream()
    .filter(s -> Character.isDigit(s.charAt(0)))
    .collect(Collectors.toList());
// ["1wood", "23bottle"]

// Q: Find second longest word in a string
String sentence = "Java streams are very powerful and flexible";
Optional<String> secondLongest = Arrays.stream(sentence.split("\\s+"))
    .distinct()
    .sorted(Comparator.comparingInt(String::length).reversed())
    .skip(1)
    .findFirst();
System.out.println(secondLongest.orElse("N/A")); // "powerful"

// Q: Sort ["file_28","file_","file_56","file_122"] numerically
List<String> files = Arrays.asList("file_28","file_","file_56","file_122");
files.sort(Comparator.comparingInt(s -> {
    String num = s.replaceAll("[^0-9]", "");
    return num.isEmpty() ? -1 : Integer.parseInt(num);
}));
System.out.println(files); // [file_, file_28, file_56, file_122]
```

Q44. Convert List of Employees to Map using Stream

```
// Two employees – convert to Map<String, Employee> (name -> employee)
List<Employee> emps = List.of(
    new Employee("Alice", "IT", 90000, 6),
    new Employee("Bob", "HR", 45000, 3)
);

Map<String, Employee> empMap = emps.stream()
    .collect(Collectors.toMap(
        Employee::name,    // key extractor
        e -> e             // value (identity)
    ));

// Handle duplicate keys
Map<String, Employee> safeMap = emps.stream()
    .collect(Collectors.toMap(
        Employee::name,
        e -> e,
        (existing, replacement) -> existing // keep existing on duplicate
    ));
```

■ Quick Reference Cheat Sheet

Collection Complexity

Collection	get(i)	add (end)	add (mid)	contains
ArrayList	O(1)	O(1)*	O(n)	O(n)
LinkedList	O(n)	O(1)	O(n)	O(n)
HashSet	N/A	O(1)	N/A	O(1)
TreeSet	N/A	O(log n)	N/A	O(log n)
HashMap	O(1)	O(1)	N/A	O(1) key
TreeMap	O(log n)	O(log n)	N/A	O(log n)
PriorityQueue	O(n)	O(log n)	N/A	O(n)

Functional Interfaces

Interface	Method	Signature	Example Use
Predicate<T>	test(T t)	$T \rightarrow \text{boolean}$	filter(isEven)
Function<T,R>	apply(T t)	$T \rightarrow R$	map(String::length)
Consumer<T>	accept(T t)	$T \rightarrow \text{void}$	forEach(println)
Supplier<T>	get()	$() \rightarrow T$	orElseGet(...)
BiFunction<T,U,R>	apply(T t, U u)	$T, U \rightarrow R$	merge((v1,v2)->v1+v2)
UnaryOperator<T>	apply(T t)	$T \rightarrow T$	replaceAll(s->s.trim())
BinaryOperator<T>	apply(T t1, T t2)	$T, T \rightarrow T$	reduce((a,b)->a+b)

Stream Terminal Operations

Operation	Returns	Example
collect(Collector)	R	collect(toList())
count()	long	stream.count()
findFirst()	Optional<T>	stream.findFirst()
min/max(Comparator)	Optional<T>	max(Comparator.naturalOrder())
anyMatch(Predicate)	boolean	anyMatch(e->e.salary()>50000)
allMatch(Predicate)	boolean	allMatch(e->e.age()>=18)
reduce(BinaryOp)	Optional<T>	reduce((a,b)->a+b)
forEach(Consumer)	void	forEach(System.out::println)
toArray()	Object[]	stream.toArray()

■ Final Tips & Common Mistakes

Topic	Remember
String comparison	Always use <code>.equals()</code> not <code>==</code> for Strings. <code>==</code> compares references.
HashMap keys	Override BOTH <code>hashCode()</code> AND <code>equals()</code> for custom objects used as keys.
Static method hiding	Static methods are NOT overridden — they are hidden. Reference type decides which is called.
ConcurrentHashMap	Does NOT allow null keys or values. HashMap allows one null key.
Fail-fast iterators	Never modify a collection while iterating with regular Iterator — use <code>Iterator.remove()</code> or <code>CopyOnWriteArrayList</code> .
Optional	Don't use <code>Optional.get()</code> without <code>isPresent()</code> check. Prefer <code>orElse/orElseGet/orElseThrow</code> .
Stream reuse	Streams cannot be reused after a terminal operation — create a new stream.
serialVersionUID	Always explicitly declare it. Auto-generated UID breaks deserialization on class change.
Spring DI	Prefer constructor injection over field injection for testability and immutability.
SOLID in interviews	Give a real project example for each principle — don't just recite definitions.
Record immutability	Records are implicitly final and their fields are final — perfect for DTOs.
rebase vs merge	Never rebase shared/public branches. Only rebase your local feature branch before PR.